

行程表 (暫定)

W	Date	Lecture	Homework	TA Class
1	09/10, 09/11	Course Information / No class - Registration	Homework 00 (Wed.)	V
2	09/17, 09/18	Git Introduction		
3	09/24, 09/25	OOP Concept	Homework 01 (Wed.)	V
4	10/01, 10/02	Introducing What C++ is		
5	10/08, 10/09	Class / Essential STL Introduction	Homework 02 (Wed.)	
6	10/15, 10/16	Lec03: Encapsulation		V
7	10/22, 10/23	Lec04: Inheritance	Homework 03 (Wed.)	
8	10/29, 10/30	Lec04: Inheritance		V
9	11/05, 11/06	Physical Hand-Written Midterm	Homework 04 (Mon.)	
10	11/12, 11/13	Physical Computer-based Midterm		
11	11/19, 11/20	Lec05: Polymorphism	Homework 05 (Wed.)	V
12	11/26, 12/27	Lec05: Polymorphism		
13	12/03, 12/04	Lec05: Polymorphism - LAB	Homework 06 (Wed.)	V
14	12/10, 12/11	Lec06: Composition & Interface		
15	12/17, 12/18	Lec06: Composition & Interface	Homework 07 (Wed.)	
16	12/24, 12/25	Lec06: Composition & Interface - Lab/ No class		V
17	12/31, 01/01	Physical Hand-Written Final / No class		
18	01/07, 01/08	Lec07: Experience sharing & OOPL - Preparation / Physical Computer-based Final		

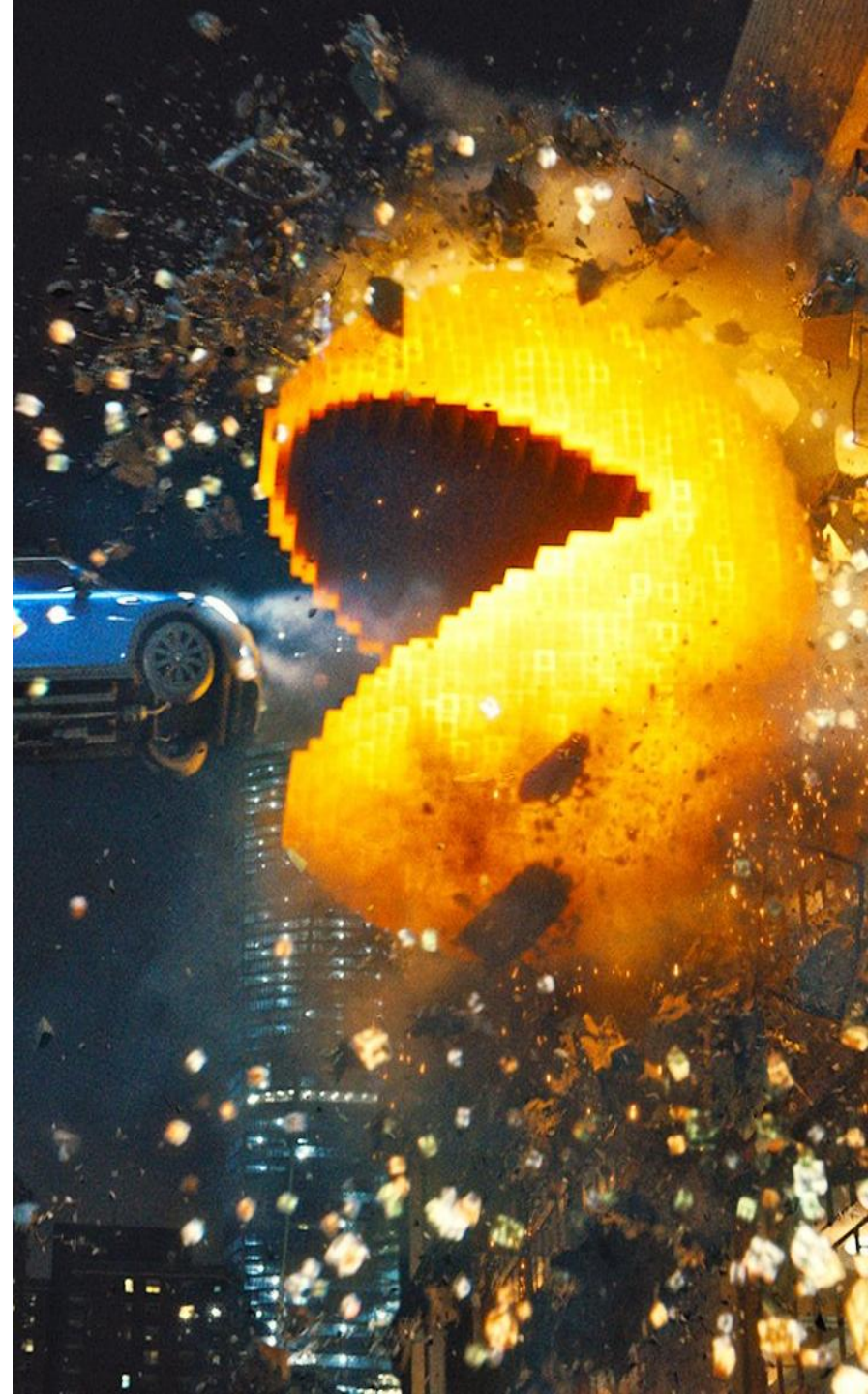
物件導向程式設計

第伍章：封裝

授課講師：孫勤昱博士

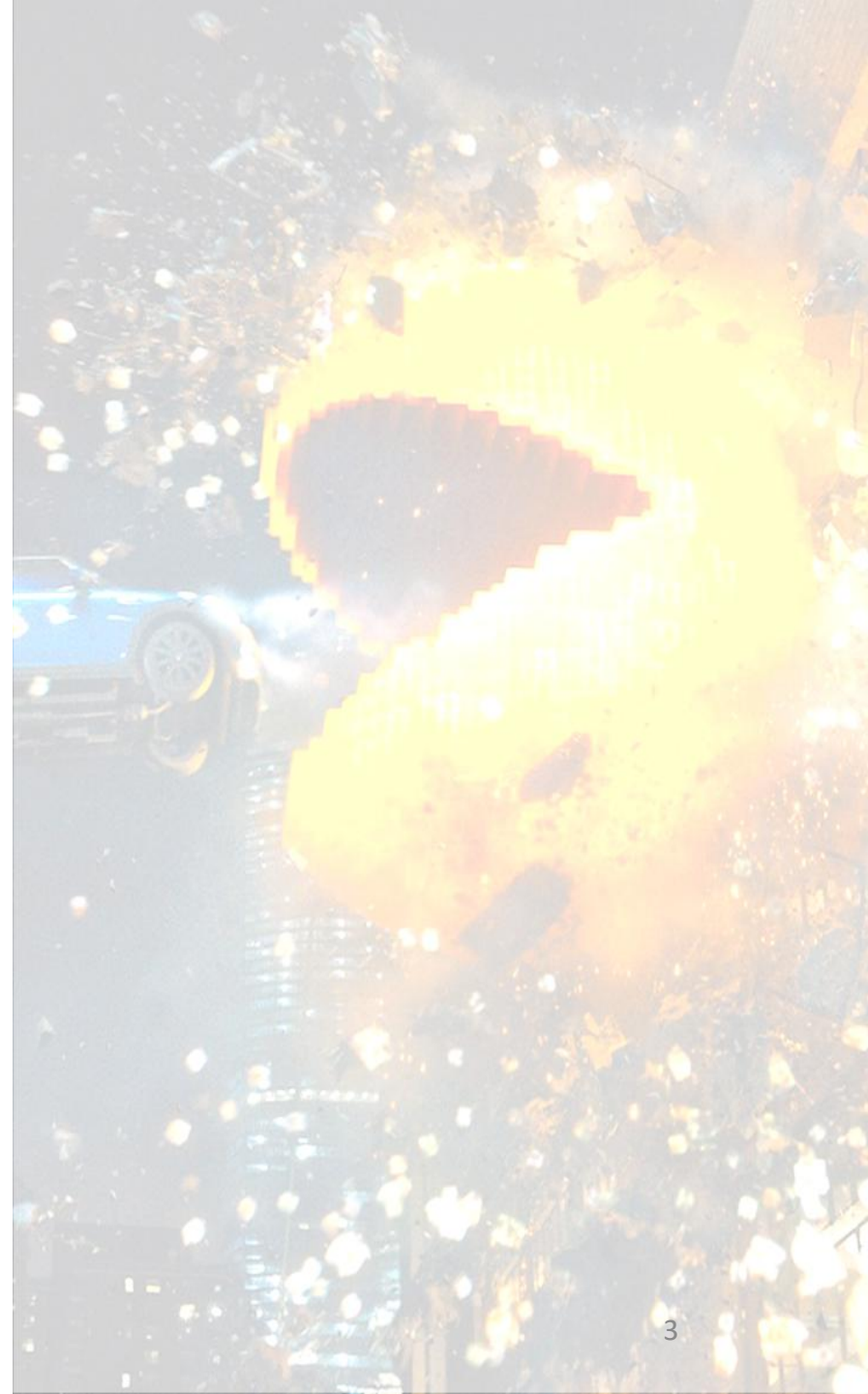
國立臺北科技大學 資訊工程系

cysun@ntut.edu.tw



大綱

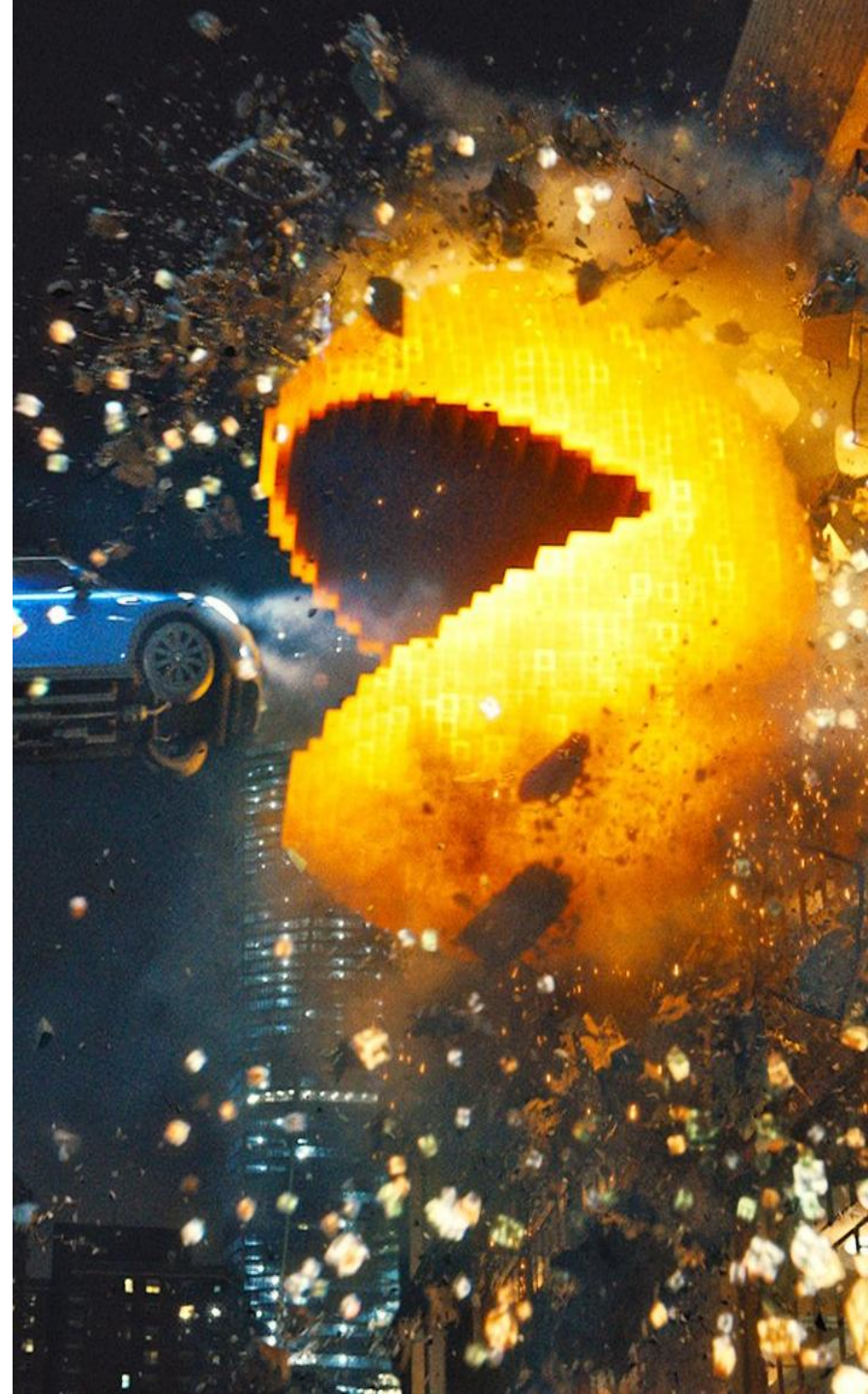
- 為什麼我們會需要封裝
- 權限控制 (Private、Public)
- Getter/Setter
- 補充資料



主題一：

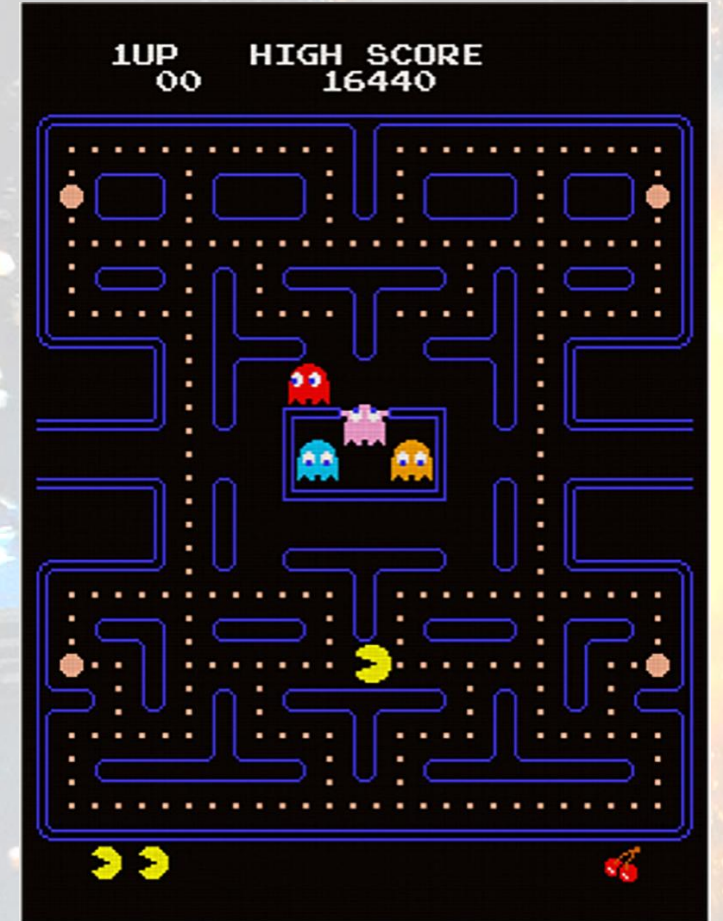
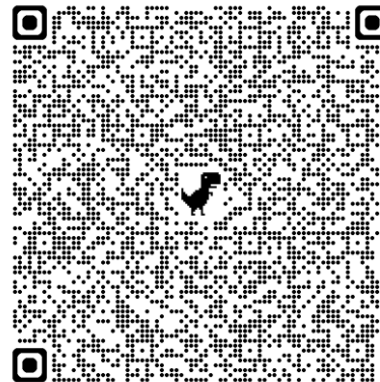
為什麼我們會需要封裝？

以 Pac-Man 遊戲為例



Pac-Man

- Bandai NAMCO製作，1980年風靡全球
- 遊戲概念：
 - 主角：Pac-Man
 - 四隻幽靈：Blinky、Pinky、Inky、Clyde
 - 豆子、特殊能力的豆子（暫時有能力吃掉幽靈）、加分水果
- 遊戲模式：
 - 在**有限次數**內，控制遊戲的主角吃掉迷宮內所有的豆子
 - 不能被幽靈抓到
 - 分數越高越厲害



使用類別來實現 Pac-Man

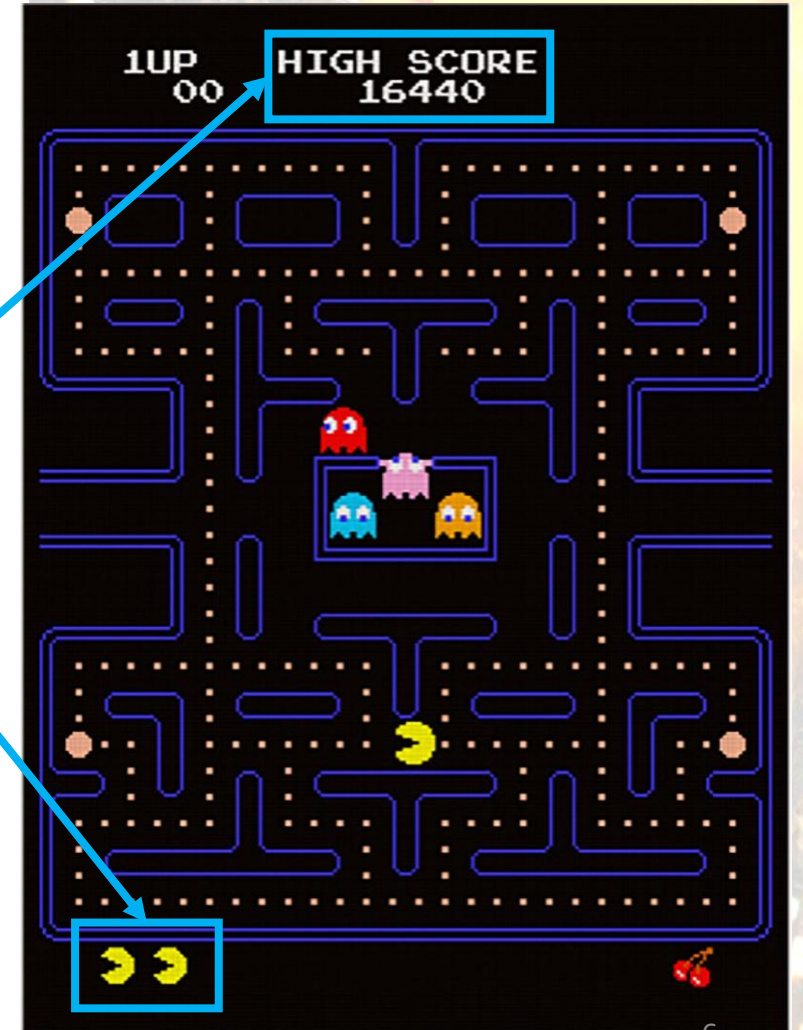
- 簡易範例 (尚未導入 Private/Public 概念)

```
Pacman.hpp

1  #ifndef PACMAN_HPP
2  #define PACMAN_HPP
3
4  #include <string>
5  #include <vector>
6
7  class Pacman {
8  public:
9      int live = 3;
10     int point = 0;
11
12     int getPoint();
13     void setPoint(int point);
14     int getLive();
15     void setLive(int live);
16 };
17
18 #endif
19
```

```
Pacman.cpp

1  #include "../include/pacman.hpp"
2
3  int Pacman::getPoint(){
4      return point;
5  }
6
7  int Pacman::getLive(){
8      return live;
9  }
10
11 void Pacman::setLive(int live){
12     this->live = live;
13 }
14
15 void Pacman::setPoint(int point){
16     this->point = point;
17 }
```



Main.cpp內容

- 假設，我們預期這個類別：
- 當被幽靈抓到時：
 - 生命值 (live) 減少一命
- 當吃到豆子時：
 - 分數 (point) 增加一百分
- 這個類別滿足了我們的需求
- 但...

```
Pacman.hpp
1  #ifndef PACMAN_HPP
2  #define PACMAN_HPP
3
4  #include <string>
5  #include <vector>
6
7  class Pacman {
8  public:
9      int live = 3;
10     int point = 0;
11
12     int getPoint();
13     void setPoint(int point);
14     int getLive();
15     void setLive(int live);
16 };
17
18 #endif
19
```

```
main.cpp
1  #include "../include/pacman.hpp"
2
3  int main(){
4      Pacman pacman;
5
6      /* 如果被鬼抓到之後 ... */
7
8      /* 第一種方法：使用直接存取成員的方式將生命值 -1 */
9      pacman.live -= 1;
10
11     /* 第二種方法：使用函數 */
12     pacman.setLive(pacman.getLive() - 1);
13
14     /* ===== */
15
16     /* 如果吃到糖豆之後 */
17
18     /* 第一種方法：使用直接存取成員的方式將分數加 100 */
19     pacman.point += 100;
20
21     /* 第二種方法：使用函數 */
22     pacman.setPoint(pacman.getPoint() + 100);
23 }
```

設計問題

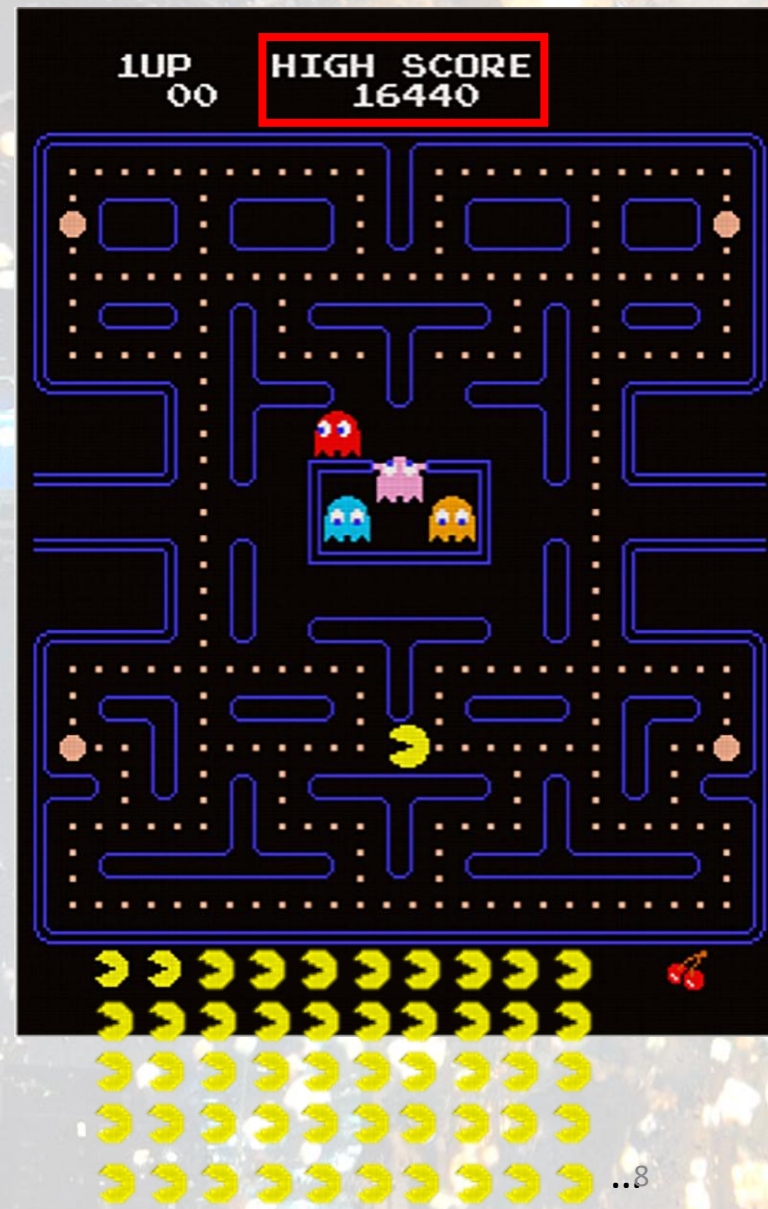
- 破壞遊戲平衡

```
Pacman.hpp

1  #ifndef PACMAN_HPP
2  #define PACMAN_HPP
3
4  #include <string>
5  #include <vector>
6
7  class Pacman {
8  public:
9      int live = 3;
10     int point = 0;
11
12     int getPoint();
13     void setPoint(int point)
14     int getLive();
15     void setLive(int live);
16 };
17
18 #endif
19
```

直接存取成員
pacman.live = 144

直接使用沒有限制的Setter
pacman.setLive(144)



解決方法

- 那麼顯然，我們必須要做一些措施來讓這件事情不會被發生
 - 不能讓成員直接被存取
 - 若要更改值，確認丟進來的值是不是合法的
 - 對值進行特殊處理法，不要讓使用者直接更改值（思考題）



主題二：

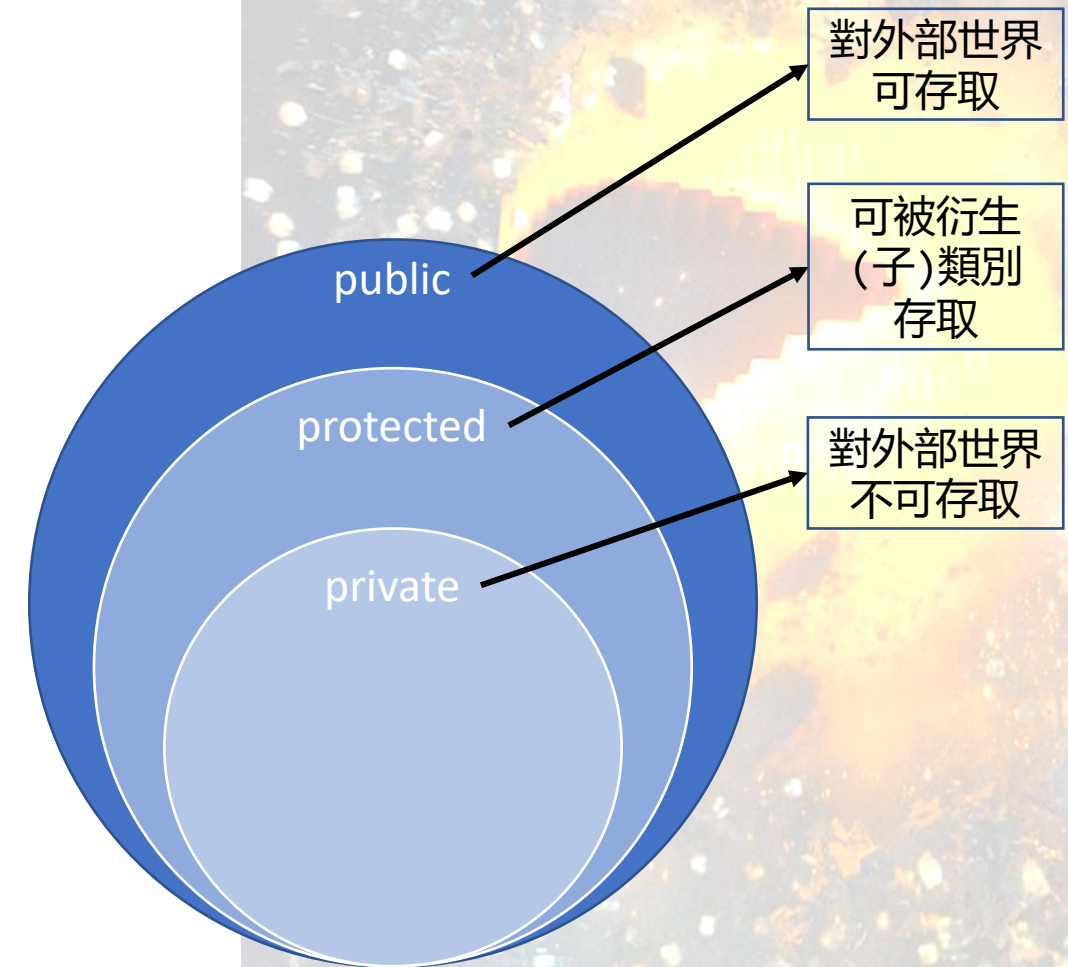
存取修飾字 (Access modifier)

Public & Private



存取修飾字

- 要做到資訊隱藏 (Information hiding) 可以使用存取修飾字
- 分為：
 - Public: 是指對存取權限完全的公開, 任何物件都可以存取
 - Private: 是限制最嚴格的access modifier, 只有在類別本身內部可以存取
 - Protected: 存取是受限的, 除了類別自己可以使用外, 只有其成員函式與類別的朋友 (friend) 以及子類別可以存取
 - 「繼承」章節的時候介紹



Case1: Public

- 為什麼我們可以直接存取成員？
 - 因為 `public` 將所有的成員能夠被外部存取
 - 在 `public` 底下的 code block 都能夠被外部存取

Pacman.hpp

```
1  #ifndef PACMAN_HPP
2  #define PACMAN_HPP
3
4  #include <string>
5  #include <vector>
6
7  class Pacman {
8  public:
9      int live = 3;
10     int point = 0;
11     int getPoint();
12     void addPoints();
13     int getLive();
14     void decreaseLive();
15 };
16
17 #endif
18
```

Case2: Private

- 另一個權限控制稱為 `private`
 - `Private` 底下的成員僅能在類別內存取，外部無法存取
- 透過右圖的修改，我們可以確保 `live` 與 `point` 無法被外部存取
 - `pacman.live` -> 編譯錯誤
 - `pacman.point` -> 編譯錯誤
- 若未指定存取修飾字，則預設是 `private`

```
Pacman.hpp

1  #ifndef PACMAN_HPP
2  #define PACMAN_HPP
3
4  #include <string>
5  #include <vector>
6
7  class Pacman {
8  private:
9      int live = 3;
10     int point = 0;
11 public:
12     int getPoint();
13     void addPoints();
14     int getLive();
15     void decreaseLive();
16 };
17
18 #endif
19
```

權限控制

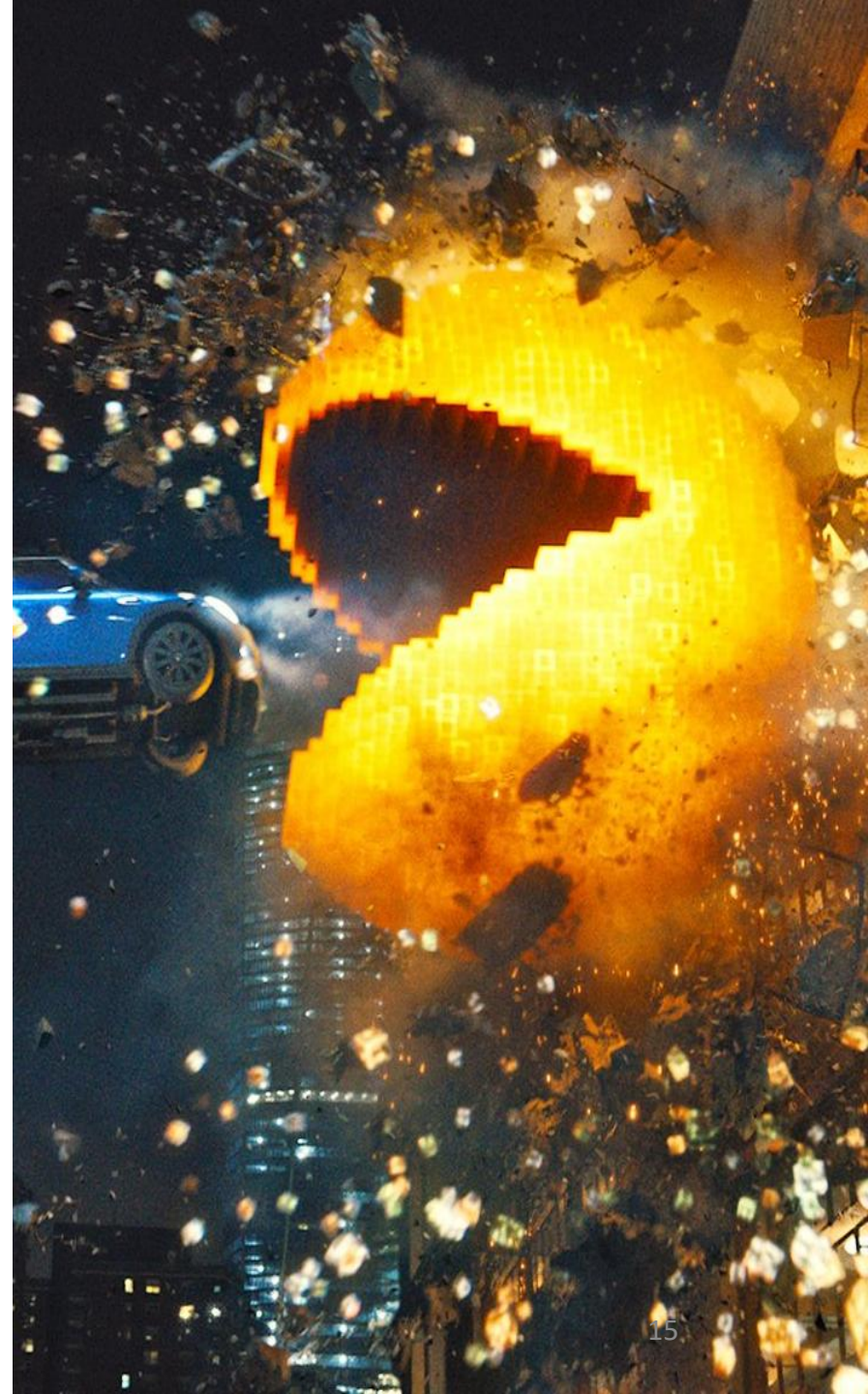
- 透過 `private` 我們可以用來使成員 **(或函式)** 無法被外部直接存取
- 透過 `public` 我們可以用來使成員 **(或函式)** 能夠被外部存取

Pacman.hpp

```
1  #ifndef PACMAN_HPP
2  #define PACMAN_HPP
3
4  #include <string>
5  #include <vector>
6
7  class Pacman {
8  private:
9      int live = 3;
10     int point = 0;
11 public:
12     int getPoint();
13     void addPoints();
14     int getLive();
15     void decreaseLive();
16 };
17
18 #endif
19
```


主題三：

Getter/Setter



Getter/Setter 的概念

- 如果不允許外部直接存取成員
 - 則需要使用**函數**來存取
 - Private屬性下，成員變數的管家（或溝通橋梁）
- 分成兩類：
 - 從物件取得資料：Getter
 - getPoint(): 取得Pacman的分數
 - getLive(): 取得Pacman還有幾條命
 - 從物件修改、存取成員、執行事件：Setter
 - addPoints(): 新增分數
 - decreaseLive(): 減少生命數量

```
1  #ifndef PACMAN_HPP
2  #define PACMAN_HPP
3
4  #include <string>
5  #include <vector>
6
7  class Pacman {
8  private:
9      int live = 3;
10     int point = 0;
11 public:
12     int getPoint();
13     void addPoints();
14     int getLive();
15     void decreaseLive();
16 };
17
18 #endif
19
```

Getter/Setter 的職責

- Getter
 - 回傳數值給Caller
- Setter
 - 檢查Caller傳入的數值會不會造成異常
 - 設定數值至成員

Caller



Request value

Return value

Provide value

Set value

```
1  #ifndef PACMAN_HPP
2  #define PACMAN_HPP
3
4  #include <string>
5  #include <vector>
6
7  class Pacman {
8  private:
9      int live = 3;
10     int point = 0;
11 public:
12     int getPoint();
13     void addPoints();
14     int getLive();
15     void decreaseLive();
16 };
17
18 #endif
19
```

Check value

檢查異常情況

- 延續前個例子，當玩家吃到道具根據道具種類加分
- 當有人呼叫 `addPoints()`，應該根據道具種類傳入加分數
 - 豆子 -> +10 分
 - 能力豆 -> +50 分
 - 水果 -> +100 分
 - 等等...
 - **傳入的分數不該為負數**

```
1  #ifndef PACMAN_HPP
2  #define PACMAN_HPP
3
4  #include <string>
5  #include <vector>
6
7  class Pacman {
8  private:
9      int live = 3;
10     int point = 0;
11 public:
12     int getPoint();
13     void addPoints(int point);
14     int getLive();
15     void decreaseLive();
16 };
17
18 #endif
19
```

Setter 的防範性

- 延續前個例子，當玩家吃到道具根據道具種類加分
- 當有人呼叫addPoints()，應該根據道具種類傳入加分數
 - 豆子 -> +10 分
 - 能力豆 -> +50 分
 - 水果 -> +100 分
 - 等等...
 - **傳入的分數不該為負數**
- 我們先省略實踐這些子函式，單純實踐看看 addPoints 函式
 - Getter的職責：將值回傳
 - Setter的職責：防範使用者傳入的值、設定值、執行事件
 - Setter防範的部分：**收到非預期的code時，跳出例外**

```
1  #include "pacman.hpp"
2
3  #include <stdexcept>
4
5  int Pacman::getPoint() {
6      return this->point;
7  }
8
9  int Pacman::getLive() {
10     return this->live;
11 }
12
13 void Pacman::decreaseLive() {
14     this->live--;
15 }
16
17 void Pacman::addPoints(int point) {
18     if (point < 0) {
19         throw std::invalid_argument("negative score is invalid");
20     }
21
22     this->point += point
23 }
24
```

Getter的職責

Setter的職責

Getter/Setter 的小結論

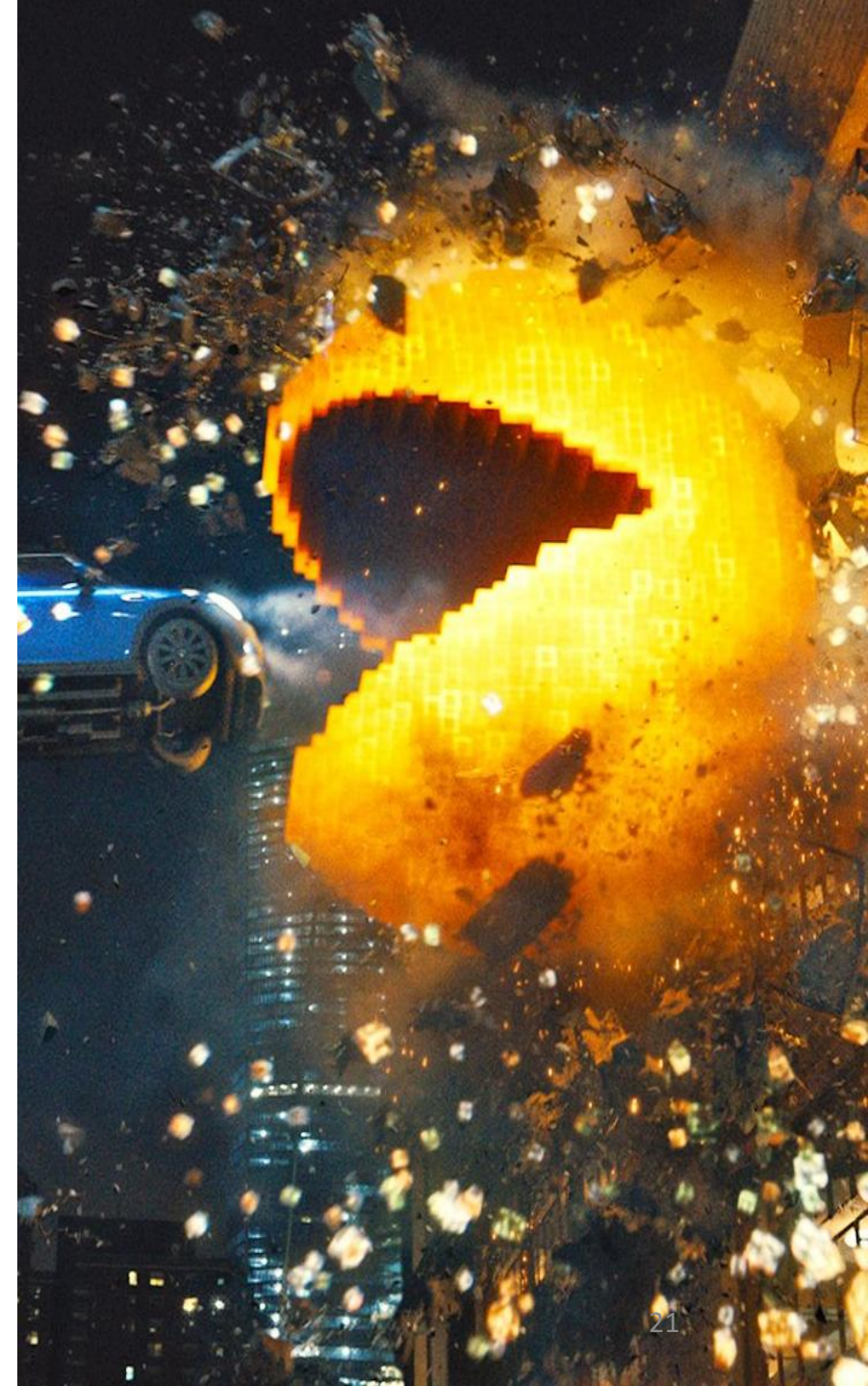
- 透過Getter/Setter,
 - 我們可以明確分責函式
 - 並且易於實踐封裝的特性
 - 進而防止異常情況發生

```
1 #include "pacman.hpp"
2
3 #include <stdexcept>
4
5 int Pacman::getPoint() {
6     return this->point;
7 }
8
9 int Pacman::getLive() {
10    return this->live;
11 }
12
13 void Pacman::decreaseLive() {
14     this->live--;
15 }
16
17 void Pacman::addPoints(int point) {
18     if (point < 0) {
19         throw std::invalid_argument("negative score is invalid");
20     }
21
22     this->point += point
23 }
24
```



主題四：

補充資料



Good Practice

- 對值進行特殊處理法，不要讓使用者直接更改值（思考題）
 - 讓 `pacman.live` 與 `pacman.point` 這樣的存取被擋下
- 在功能上限制：將 `setLive()` 與 `setPoint()` 使用 `decreaseLive()` 與 `addPoints()` 所替代
- 讓所有異常行為能夠被防範，以及能夠依然滿足需求

```
1  #ifndef PACMAN_HPP
2  #define PACMAN_HPP
3
4  #include <string>
5  #include <vector>
6
7  class Pacman {
8  private:
9      int live = 3;
10     int point = 0;
11 public:
12     int getPoint();
13     void addPoints();
14     int getLive();
15     void decreaseLive();
16 };
17
18 #endif
19
```

`pacman.hpp`

```
1  #include "../include/pacman.hpp"
2
3  int Pacman::getPoint(){
4      return point;
5  }
6
7  int Pacman::getLive(){
8      return live;
9  }
10
11 void Pacman::decreaseLive(){
12     this->live -= 1;
13 }
14
15 void Pacman::addPoints(){
16     this->point += 100;
17 }
```

`pacman.cpp`

Bonus: 如何檢驗封裝是好的

- 會不會有異常情況沒有被考慮到?
- **撰寫 (單元) 測試**
 - 邊界檢查
 - 非法值
 - 可能還要有一點通靈成分
- **換位思考**
 - 不要只站在寫這個封裝的人角度思考，而要站在「使用這個封裝的人」角度來檢驗它
 - 這個封裝有沒有準確的建模問題領域 (Problem Domain)?
 - 你的系統與封裝是為了解決什麼樣的問題?
 - 它在「哪一個世界」中運作? 那個世界有哪些重要概念 (名詞) 與行為 (動詞)?

```
ut_pacman.cpp
1  #include <gtest/gtest.h>
2
3  #include "../include/pacman.hpp"
4
5  TEST(PacmanTest, UseDecreaseLiveShouldDecrease1Live){
6      Pacman pacman;
7      int live = pacman.getLive();
8
9      pacman.decreaseLive();
10
11     ASSERT_EQ(pacman.getLive(), live - 1);
12 }
13
14 TEST(PacmanTest, UseAddPointsShouldAdd100Points){
15     Pacman pacman;
16     int point = pacman.getPoint();
17
18     pacman.addPoints();
19
20     ASSERT_EQ(pacman.getPoint(), point + 100);
21 }
```

補充：思考好的封裝

- 如果使用 enum class
 - 不需要特判(exception)
 - 使用者不需要擔心傳入錯誤的數值
 - 如果遊戲邏輯更改，可以在統一的地方修改

```
1  #ifndef PACMAN_HPP
2  #define PACMAN_HPP
3
4  #include <string>
5  #include <vector>
6
7  class Pacman {
8  private:
9      int live = 3;
10     int point = 0;
11 public:
12     enum class ScoreItem {
13         PacDot,
14         PowerPellet,
15         Cherry,
16         // ...
17     };
18
19     int getPoint();
20     void addPoints(ScoreItem item);
21     int getLive();
22     void decreaseLive();
23 };
24
25 #endif
26
```

```
1  #include "pacman.hpp"
2
3  int Pacman::getPoint() {
4      return this->point;
5  }
6
7  int Pacman::getLive() {
8      return this->live;
9  }
10
11 void Pacman::decreaseLive() {
12     this->live--;
13 }
14
15 void Pacman::addPoints(ScoreItem item) {
16     switch (item) {
17     case ScoreItem::PacDot:
18         this->score += 10;
19         break;
20     case ScoreItem::PowerPellet:
21         this->score += 50;
22         break;
23     case ScoreItem::Cherry:
24         this->score += 100;
25         break;
26     // no need for 'default'
27     }
28 }
29
```


補充: struct

- C++ 的 Class 與 Struct 都可以:
 - 宣告成員變數與成員函式
 - 繼承、多型、建構子、介面
- 差異:
 - Struct 預設屬性為 Public; Class 預設屬性為 Private
 - Struct 預設 Public 繼承; Class 預設 Private 繼承
 - Struct 不能用在 template; Class 可以用在 template
- 使用時機:
 - 單純只有資料, 不會針對內部成員做複雜處理的時候, 使用 struct
 - 其他時候都使用 class

```
1 #include <iostream>
2
3 // 定義一個名為 Point 的結構
4 struct Point {
5     int x; // x 座標
6     int y; // y 座標
7
8     // 建構函數
9     Point(int xVal, int yVal) : x(xVal), y(yVal) {}
10
11     // 成員函數用於顯示點的座標
12     void display() const {
13         std::cout << "Point(" << x << ", " << y << ")\n";
14     }
15 };
16
17 int main() {
18     Point p1(5, 7); // 建立一個 Point 物件並初始化其 x 和 y 值
19     p1.display();   // 輸出: Point(5, 7)
20
21     return 0;
22 }
```

```
template<typename T> 可以這樣
class Array1 {};

template<class T> 可以這樣
class Array2 {};

template<struct T> 不可以這樣
class Array3 {};
```

封裝

(Source: [臺大開放式課程](#))

- 類別將成員變數與成員函式封裝起來
 - 當作一個整體來思考運用
 - 使類別內容的修改更容易
 - 把 `class` 變成一個黑箱 (Black box) 模組
 - `Programmer` 不用瞭解成員變數與成員函式的細節, 就能應用類別的功能
 - 程式碼重複使用 (Code reuse)

Private



封裝

(Source: [臺大開放式課程](#))

- 硬體 IC 的封裝
 - 内部的電路封裝在內，不輕易顯露
 - 電子電路工程師只要在接腳施加正確的電壓電流訊號，IC 就能正常運作
 - 大大減輕電子電路工程師的負擔
- 軟體類別上的封裝
 - 軟體工程師不需要了解 `private` 成員變數與成員函式
 - 只要懂得 `public` 成員的使用，也能正確應用既有類別程式
 - 減輕工作負荷，增進效能



https://en.wikipedia.org/wiki/Integrated_circuit

封裝

(Source: [臺大開放式課程](#))

- 通常成員變數為 **private**
 - 只供同類別之功能函式取用
 - 不可以由類別外直接取用
- 若有必要讓外界取得或設定其值
 - 通常另外再寫 **public** 成員函式處理
 - 避開同名變數問題
 - 避免被不慎改動，破壞狀態的一致性
 - 容易維護，不影響外界程式
- 功能函式可能 **Public** 亦可能 **Private**
- 一個類別至少有一個公開函式，以便應用



封裝

(Source: [臺大開放式課程](#))

```
1 public int width;  
2 public int height;  
3 public int area;  
4  
5 Rectangle a = new Rectangle();  
6 a.width = 3;  
7 a.height = 5;  
8 a.area = a.width * a.height;
```



- 問題(一): 程式邏輯中, 或許實作面積計算的 code 不需要給別人看不想 (保護細節)
- 問題(二): 程式碼的 Area 計算方式, 或許不是 $\text{width} * \text{height}$

封裝

(Source: [臺大開放式課程](#))

```
1 private int width;  
2 private int height;  
3 private int area;  
4  
5 Rectangle a = new Rectangle();  
6 a.width = 3;  
7 a.height = 5;  
8 a.area = a.width * a.height;
```



- 若改成 `private`
 - Line 6 - 8 編譯不會過



width

height



封裝

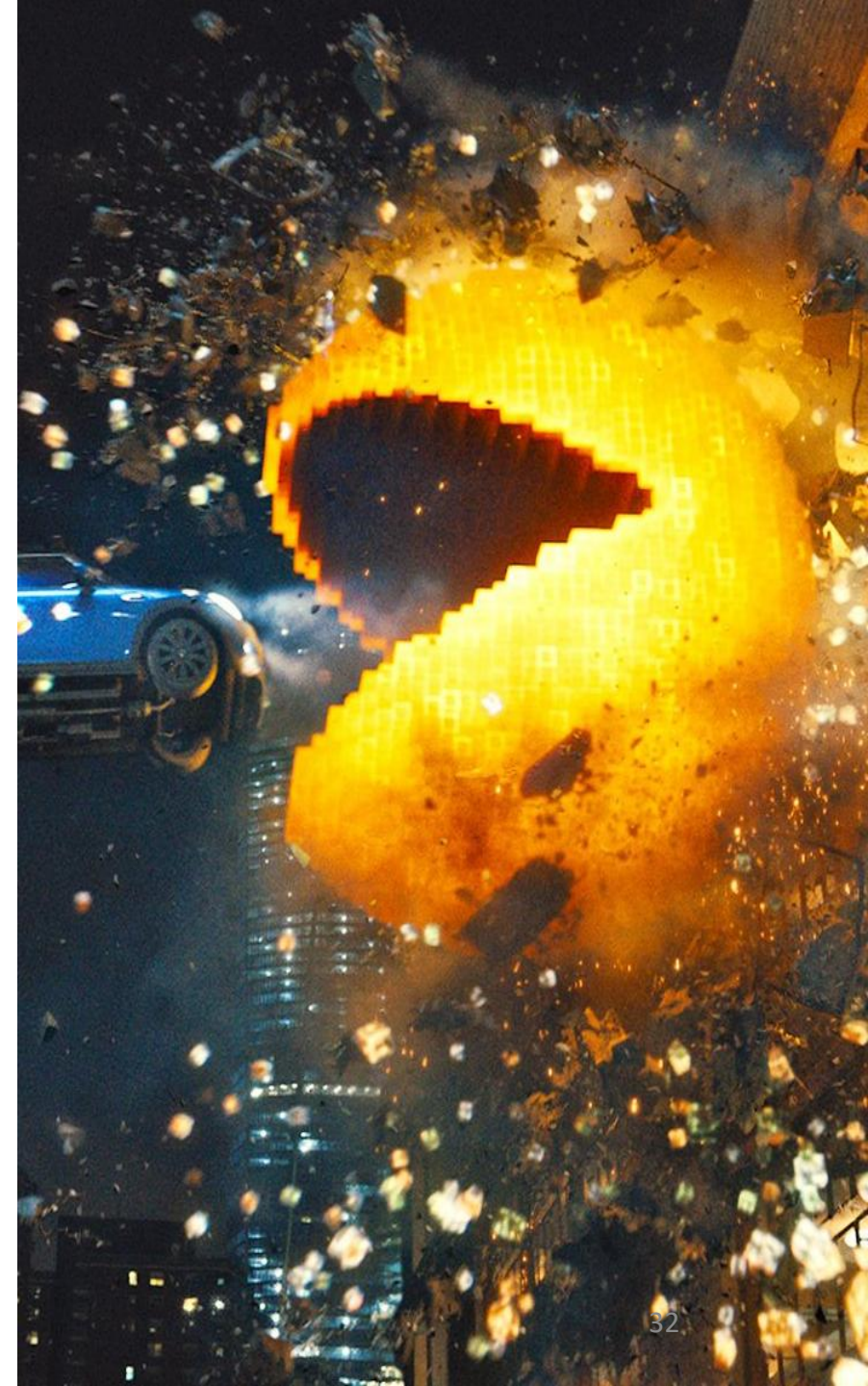
(Source: [臺大開放式課程](#))

```
1 private int width;
2 private int height;
3 private int area;
4
5 public void SetWidth(int w) {
6     width = w;
7 }
8
9 public void SetHeight(int h) {
10     height = h;
11 }
12
13 public int GetArea() {
14     area = width * height;
15     return area;
16 }
17
18 Rectangle a = new Rectangle();
19 a.SetWidth(3);
20 a.SetHeight(5);
21 a.GetArea();
```

- 要新增 Setter/Getter
- 不能有 SetArea
 - 避免被 User 破壞狀態的一致性
- 使用 enum class 或多型來處理不同形狀的面積計算

主題五：

小總結與討論



小總結

- 思考到底要讓他人怎麼來使用這個 `class`
 - 思考哪些 `member` 需要保護
 - 是不是所有的 `member` 都需設計一個 `getter` 與 `setter`
 - 是不是有其他更方便他人使用的 `method`



開放式問題：Member 到底需不需要都設 private?

- Member 是否需要保護?

```
main.cpp
1  #ifndef PACMAN_HPP
2  #define PACMAN_HPP
3
4  class Pacman{
5      private:
6          int a;
7          int b;
8          int c;
9          int d;
10     public:
11         void setA(int live);
12         int getA();
13         void setB(int live);
14         int getB();
15         void setC(int live);
16         int getC();
17         void setD(int live);
18         int getD();
19 };
20
21 #endif
```

```
main.cpp
1  #ifndef PACMAN_HPP
2  #define PACMAN_HPP
3
4  class Pacman{
5      public:
6          int a;
7          int b;
8          int c;
9          int d;
10
11         void setA(int live);
12         int getA();
13         void setB(int live);
14         int getB();
15         void setC(int live);
16         int getC();
17         void setD(int live);
18         int getD();
19 };
20
21 #endif
```

```
main.cpp  pacman.hpp  pacman.cpp
1  #include "pacman.hpp"
2  #include <iostream>
3
4  int main()
5  {
6      Pacman A;
7
8      //Case 1
9      int Counter = A.getA();
10     Counter += 1;
11     A.setA(Counter);
12     std::cout << A.getA() << std::endl;
13
14
15
16
17
18 }
```

開放式問題： Getter/Setter 是否必要？

```
1 class Player {
2     public:
3         int getX();
4         int getY();
5
6         void setX(int x);
7         void setY(int y);
8
9     private:
10        int x;
11        int y;
12 };
13
14 int main(void){
15     Player player;
16
17     // 玩家移動
18     int x = player.getX();
19     x += 0;
20     player.setX(x);
21
22     int y = player.getY();
23     y += 1;
24     player.setY(y);
25 }
```

```
1 class Player {
2     public:
3         // int getX();
4         // int getY();
5
6         // void setX(int x);
7         // void setY(int y);
8
9     void move(int x, int y){
10        this->x += x;
11        this->y += y;
12    }
13
14     private:
15        int x;
16        int y;
17 };
18
19 int main(void){
20     Player player;
21
22     // 玩家移動
23     player.move(0, 1);
24 }
```

